

LLM 和高级 RAG 的 Python 程序设计智能问答系统构建

孙彦武¹, 王赠凯², 刘宏兵¹, 王 薇¹, 姜梦琦¹

(1. 嘉兴南湖学院 信息工程学院, 浙江 嘉兴 314001;

2. 嘉兴大学 人工智能学院, 浙江 嘉兴 314001)

摘 要:针对知识幻觉与时效滞后问题对大语言模型在教育应用中的影响,利用大模型和检索增强生成技术构建了基于课程知识库的教学辅助智能问答系统。该系统基于 Streamlit 和 LangChain 框架,运用自适应权重分配算法动态调整 BM25 关键词检索与向量语义检索的融合比例,结合上下文重排序技术优化大语言模型先验知识与检索内容的重组策略,实现答疑服务与个性化学习需求的精准匹配。测试结果表明,在保障数据隐私的前提下,系统性能指标 BLEU、ROUGE-L 和准确率分别提升了 10.3%、12.10% 和 11.20%,大幅提高了 Q&A 系统问题解答的准确性和可解释性,有效强化学生的知识建构、分析问题和自主学习能力的培养。

关键词:智能问答系统; 大语言模型; 检索增强生成; Python 程序设计

中图分类号: G 482; TP 181

文献标志码: A

文章编号: 1006-7167(2026)06-0000-00



Building an Intelligent Question Answering System for Python Programming Based on LLM and Advanced RAG

Sun Yanwu¹, Wang Zengkai², Liu Hongbing¹, Wang Wei¹, Jiang Mengqi¹

(1. School of Information Engineering, Jiaxing Nanhu University, Jiaxing 314001, Zhejiang, China;

2. School of Artificial Intelligence, Jiaxing University, Jiaxing 314001, Zhejiang, China)

Abstract: To address the impact of knowledge illusion and time lag on the application of large language models in education, a teaching-aid intelligent question-answering system based on a course knowledge base is constructed by using large models and retrieval-enhanced generation techniques. Based on the Streamlit and LangChain frameworks, the system dynamically adjusts the fusion ratio of BM25 keyword retrieval and vector semantic retrieval using an adaptive weight allocation algorithm. It also optimizes the reorganization strategy of prior knowledge and retrieval content from the large language model using context reordering technology, achieving precise matching between Q&A services and personalized learning needs. Testing results show, under the premise of ensuring data privacy, the system performance metrics, i. e., BLEU, ROUGE-L, and accuracy, are increased by 10.3%, 12.10%, and 11.20%, respectively, significantly enhancing the accuracy and interpretability of the Q&A system's question answering, effectively strengthening the students' knowledge construction, problem analysis, and self-learning abilities.

Key words: intelligent question-answering system; large language model; retrieval-augmented generation;

收稿日期: 2025-12-15

基金项目: 浙江省高等教育学会高等教育研究课题 (KT2025441); 浙江省高等教育学会实验室工作分会高校实验室工作研究项目 (YB202512); 浙江省教育厅一般科研项目 (Y202558657)

作者简介: 孙彦武 (1986—), 男, 甘肃庆阳人, 博士, 实验师, 主要研究方向为机器学习, 实验室建设与管理。

Tel.: 0573-83628361; E-mail: 202386@jxnhu.edu.cn

通信作者: 王赠凯 (1980—), 男, 河南郑州人, 博士, 副教授, 主要研究方向为智能优化与学习。

Tel.: 13736409428; E-mail: wangzengkai@zjxu.edu.cn

0 引言

人工智能技术的范式革新正以前所未有的速度重塑教育生态。以大语言模型 (large language model, LLM) 为代表的生成式人工智能 (generative artificial intelligence, GAI)^[1] 凭借其强大的语义理解与生成能力,正在推动问答 (question answering, Q&A) 系统从基于规则的专家系统^[2] 向数据驱动的智能推理模式跃迁。然而,这类模型对训练数据的强依赖性导致其在生成内容时面临“知识幻觉”与“时效滞后”的双重困境^[3,4]。潜在的幻觉问题极大地限制了大语言模型在教育等实际场景中的应用,甚至衍生出法律和伦理风险^[5]。

随着教育智能化转型不断深入,“师-生-机”三元协同模式的重要性越发凸显^[6]。该模式要求问答系统不仅需要拥有专业级的学科知识,还需要支持个性化的学习路径规划以及教学策略适配。但现有通用大语言模型在专业课程中出现的知识碎片化、推理过程“黑箱化”等问题,较为严重地限制了其教学辅助作用^[7]。此外,教育数据的安全以及隐私保护等也成为学术界关注的重点,本地化模型部署和可控知识接入则成为安全可靠的解决办法。

针对上述问题,本文以“Python 语言程序设计”课程为研究载体,构建基于 LLM 与高级检索增强生成 (retrieval augmented generation, RAG) 的垂直领域 Q&A 系统。研究主要贡献包括:

- (1) 融合课程知识库和大语言模型先验知识,缓解大语言模型生成内容的幻觉问题和时效滞后困境。
- (2) 采用自适应权重分配算法,动态调整 BM25 关键词检索与向量语义检索的融合比例实现混合检索,突破单一检索增强的信息覆盖局限。
- (3) 依托本地化部署与可控知识接入,实现教学数据的安全管控。

1 系统设计与关键技术

1.1 系统框架

基于 LLM 和高级 RAG 的 Python 语言程序设计教学辅助智能 Q&A 系统的设计框架如图 1 所示。

由图 1 可见, Q&A 系统主要包含开源大模型、开源应用框架、知识库与检索增强生成和问答交互。

开源大模型 主要涵盖大语言模型、嵌入模型和重排模型,实现知识库内容嵌入、检索重排与上下文融合、问答推理和知识生成。

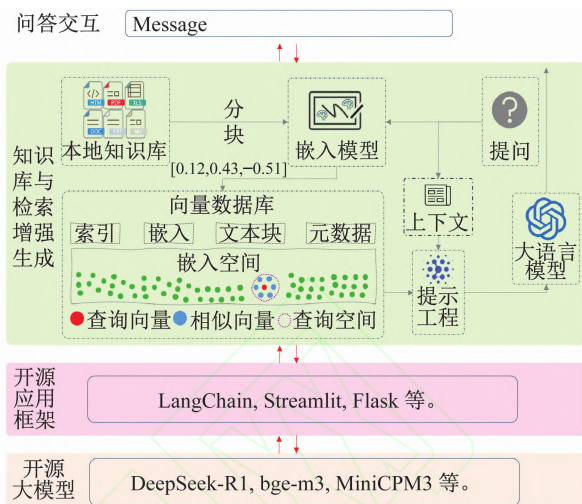


图 1 Q&A 系统设计框架

开源应用框架 用于实现基于开源大模型的知识检索和模型推理的链式工作流以及系统交互界面。

知识库与检索增强生成 用于实现知识库构建、知识检索与上下文融合和提示词工程等。

问答交互 用于实现学生提问和系统答复的信息交互。

1.2 大模型

知识幻觉是大语言模型在实际应用中面临的主要问题^[8]。“幻觉”是指生成与提供源不符或毫无意义的内容^[9]。Zhang 等^[10]认为 LLM 中的知识幻觉可分为输入冲突幻觉, LLM 生成的内容与用户提供的源输入存在偏差;语境冲突幻觉, LLM 生成的内容与先前生成的信息相冲突;事实冲突幻觉, LLM 生成的内容不忠实于既定的世界知识。针对大语言模型的幻觉问题, Kazlaris 等^[11]在综述中整理了 30 余种缓解大语言模型幻觉问题的技术,尤其是 RAG、提示工程、自然语言推理链和验证链。针对大语言模型在处理语言、知识、推理、数学、代码和指令跟随等任务的效果不一, Zhu 等^[12]提出评估大语言模型生成内容的能效指标,包括完整性、幻觉和不相关性,并运用这些指标对一系列主流大语言模型进行了评估,结果表明,中小型开源模型 MiniCPM3-4B 在中文语义理解方面性能优异,促进大语言模型的本地化部署与应用。

嵌入模型 (embedding model) 是一种能将离散符号, 比如单词、句子、图像等映射到连续向量空间的机器学习模型^[13]。目前, 主流嵌入模型在中、英跨语言检索任务的性能对比如图 2 所示 (数据来源 <https://huggingface.co/openbmb/MiniCPM-Embedding-Light>)。

由图 2 可知, 嵌入模型 MiniCPM 系列的性能优于其他模型。

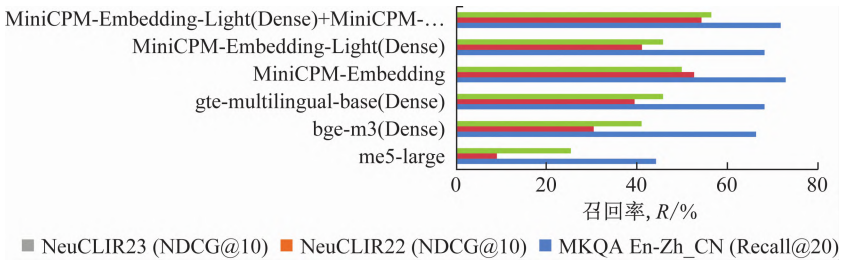


图2 主流嵌入模型在中、英跨语言检索任务的性能对比

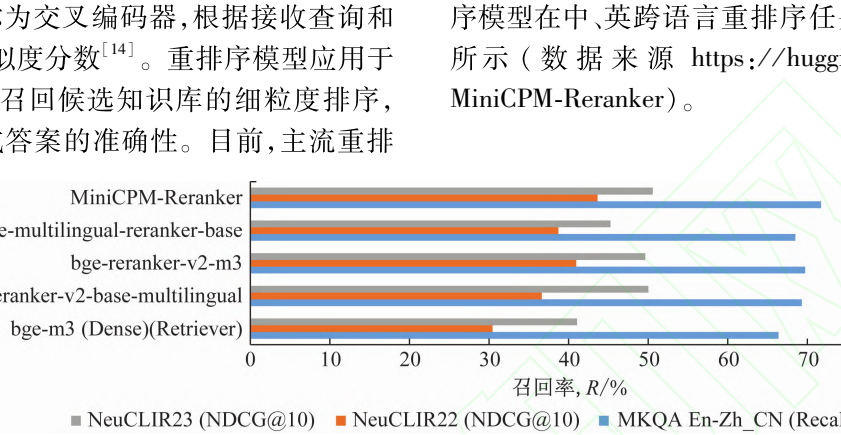


图3 主流重排序模型在中英跨语言检索任务的性能对比

重排序模型被称为交叉编码器,根据接收查询和文档对输入、输出相似度分数^[14]。重排序模型应用于 RAG 流程,旨在优化召回候选知识库的细粒度排序,提升大语言模型生成答案的准确性。目前,主流重排

序模型在中、英跨语言重排序任务的性能对比如图 3 所示(数据来源 <https://huggingface.co/openbmb/MiniCPM-Reranker>)。

由图 3 可知,重排序 MiniCPM 系列的性能优于其他模型。

1.3 检索增强生成技术

检索增强生成技术是一种结合大语言模型和信息检索的创新方法^[15]。Fan 等^[16]研究发现,检索增强生成技术能利用可靠权威且新的外部知识库有效缓解大语言模型的幻觉问题和内部知识过时的困境,提升大语言模型生成内容的质量。

基于 LLM 的 RAG 发展范式主要有朴素 RAG 和高级 RAG。由于朴素 RAG 在检索、生成和增强等方面存在的精度不高、抗干扰能力弱等问题,高级 RAG^[17]引入索引、查询和检索后过程等策略,提升了检索增强生成结果的准确性和可靠性。

1.4 应用框架

支持 LLM 和 RAG 的应用框架有 LangChain, LlamaIndex 和 FastRAG 等^[18]。LangChain 是一个辅助快速构建和部署基于外部知识库和大语言模型的开源人工智能应用框架,因其模块化架构和链式工作流,使得 LangChain 已经成为当前人工智能应用生态中的主流^[19]。系统运用 LangChain 框架实现外部知识库嵌入、大语言模型调用和推理部署,运用 Streamlit 框架^[20]实现应用交互。

2 知识库构建与检索

2.1 知识库构建

依据“Python 语言程序设计”课程^[21]的理论和实验教学内容构建知识库。理论部分涵盖 Python 语言概述及基础、流程控制、序列字典、函数、文件、面向对

象程序设计、标准库应用和第 3 方库应用等。实验部分涵盖 PTA 平台的选择题、判断题、程序填空题、函数题和编程题等题目。

(1) 知识库文档加载。通过知识库管理模块实现文件上传,运用 LangChain 的 document_loaders 模块加载上传的知识库文档,支持 pdf、txt 等文档类型。

(2) 知识库文档分割。知识库文档分割是把不同格式的文档数据分割为更小、可处理的块,以更好适应 LLM 的语境限制^[22]。系统调用 LangChain 提供的文本分割器 RecursiveCharacterTextSplitter 实现知识库文档的文本切块,文本块字符长度 chunk_size 和相邻块重叠字符数 chunk_overlap 支持可视化自定义赋值。separators = ["\n\n", "\n", ".", "。", "!", "?", ":", ";", "。", "。"]。

(3) 知识库文档向量化。知识库文档向量化是把切割的知识库 chunks 转换成向量并存储于向量数据库,即知识库文档的数值表示。开源向量数据库有 Chroma、Faiss 等。Faiss 在实时响应和最低开销方面性能更优,但 Chroma 的磁盘存储和索引功能为内存受限的大型数据集提供了更好方案。利用 Chroma 向量存储包装器实现知识库文档的存储嵌入向量和高效相似性搜索,利用 Hugging Face Embeddings 嵌入模型类实例化本地嵌入模型。引入 MiniCPM-Embedding 嵌入模型,实现知识库文本向高维向量转化,并快速计算语义相似性。

2.2 知识检索与上下文融合

(1) 混合检索与重排序。运用 BM25 关键词检索器和向量检索器相融合的混合检索机制,提出基于提

问内容的检索器配比动态调整方法。BM25 关键词检索器和向量检索器的配比与提问关键词的对应关系见表 1。

表 1 检索器占比与提问关键词的对应关系

提问内容	BM25 关键词检索器和向量检索器配比
默认	4:6
关键词查询,例如:“定义” in question or “什么是” in question,……	7:3
语义查询,例如“比较” in question or “优缺点” in question,……	3:7

知识检索与上下文融合流程如图 4 所示,经过多轮实验,混合检索与重排序检索的比重设为 3:7 时效果更好。基于图 3 数据分析结果,系统运用重排序模型 MiniCPM-Reranker。

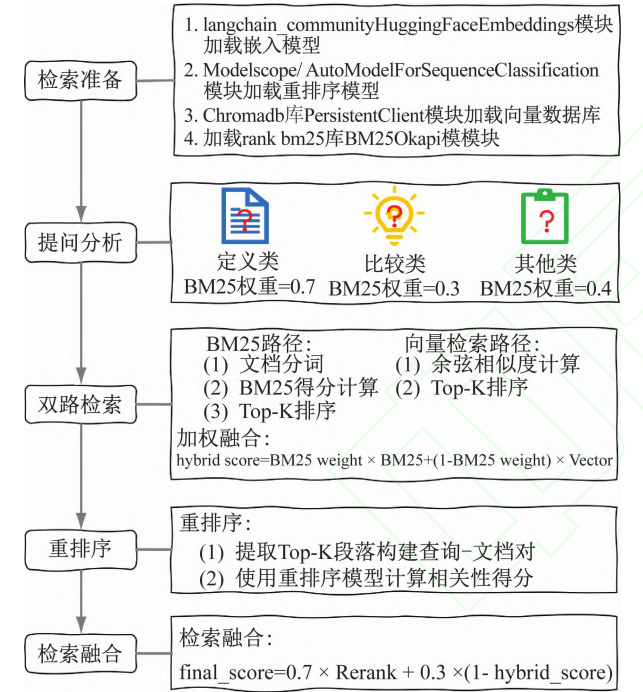


图 4 知识检索与上下文融合

(2) 上下文融合。为规避大语言模型在生成内容过程中容易出现的“信息过载”或“营养不良”等问题,系统采用上下文融合机制,通过提取混合检索和重排序确定的高相关 Chunks 信息,实现高相关性 Chunks 内容的有效融合,提高上下文内容的全面性和准确性。

2.3 提示词工程

提示词工程是改进和指导大语言模型完成生成任务的提示或指令,以指导用户与人工智能应用有效交互^[23]。系统提示词工程代码如图 5 所示。

2.4 学生提问与大语言模型智能回答

在知识库构建、混合检索与重排序、上下文融合和提示词工程基础上,引入 LLM 的推理生成能力实现智能回答。

```
SYSTEM_PROMPT = ""
你是一个知识丰富的智能助手,能够根据提供的上下文信息回答问题。
请仔细分析上下文中的相关内容,并结合问题进行准确回答。
如果上下文没有提供足够信息,请回答"根据现有信息无法确定"。
回答格式:
基于检索的知识库内容,{user_query}的答案是:
"""

def build_prompt(user_query, history=None):
    if history is None:
        history = []

    # 检索
    context = hybrid_search(user_query,
        'qa_vectors', TOP_K, RANK_TOP_K)
    messages = [{"role": "system", "content":
        SYSTEM_PROMPT}]

    # 添加对话历史
    for msg in history:
        messages.append({"role": msg["role"],
            "content": msg["content"]})

    # 添加当前问题和上下文
    user_content = f"""
        问题: {user_query}
        检索内容: {context}
        """
    messages.append({"role": "user", "content":
        user_content})

    # 引用问答模板
    prompt_text = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True,)

    return prompt_text
```

图 5 提示词工程

为降低系统部署与运算成本,利用 MiniCPM4-8B 轻量化大语言模型实现学生提问的精准理解与推理生成。如图 6 所示,系统根据学生的提问开启会话管理,在解析提问内容的基础上,运用混合检索和重排序技术实现上下文融合,根据提示词工程整合提问会话历史、检索内容上下文和大语言模型先验知识并生成回答内容,对初步生成的回答内容进行特殊标记过滤,生成最终回答,并将其加载到会话管理以更新会话历史,最后显示回答内容。

3 系统部署与能效评估

3.1 系统部署

根据大语言模型 MiniCPM 系列、嵌入模型 MiniCPM-Embedding、重排序模型 MiniCPM-Reranker 和其他组件的运行需求,系统部署环境的主要依赖见表 2。

通过安装 Anaconda,构建基于 Python 3.10 的虚拟环境 llm,在该环境下使用“streamlit run qa.py”命令启动系统。

系统不仅支持简单知识问答,还兼具复杂编程问答。简单知识问答示例如图 7 所示。

由图 7 可知,学生提问“到目前为止,程序设计语言的发展经过了哪些阶段?”,系统根据混合检索和重排序机制,从知识库中召回相关性从高到低的 5 个

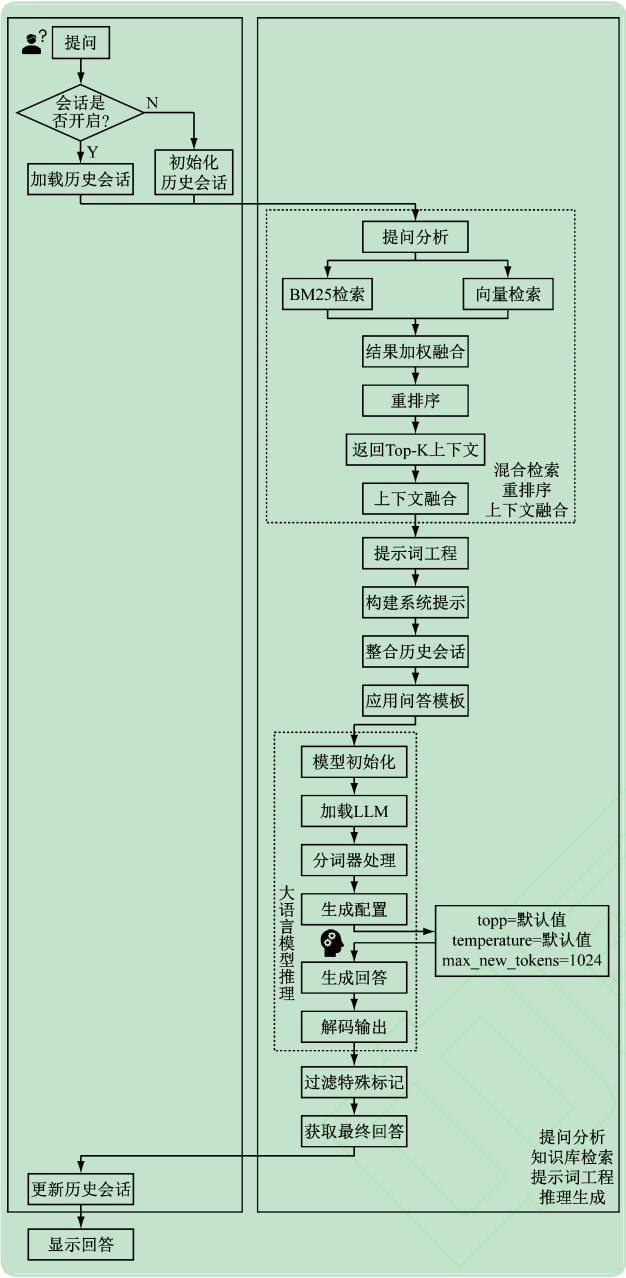


图6 学生提问与大语言模型智能回答流程图

表2 系统部署环境的主要依赖

依赖名称	版本或参数
GPU RAM	16 GB 以上/单张,按需增配
cuda	12.8
torch	2.7.1
transformers	4.52.4
modelscope	1.27.1
langchain-community	0.3.25
langchain-core	0.3.66
langchain-text-splitters	0.3.8
chromadb	1.0.13
rank-bm25	0.2.2
streamlit	1.46.0

基于LLMs和RAG的Python程序设计智能问答系统

本地大语言模型: MiniCPM-4B-8B
本地嵌入模型: MiniCPM-Embedding
本地重排序模型: MiniCPM-Reranker

到目前为止, 程序设计语言的发展经过了哪些阶段?

根据你的查询“到目前为止, 程序设计语言的发展经过了哪些阶段?”, 检索到以下相关信息:

1. 相关性得分: 0.9318
内容: 第一章 Python语言概述1.1程序设计语言及其发展一般地, 程序就是供计算机执行、能完成特定功能的指令序列。程序设计的任务就是设计解决问题的方法和步骤, 并将解决问题的方法和步骤用程序设计语言来描述。通常是先对问题进行分析并建立数学模型, 然后考虑数据的组织方式和算法, 并用某一种程序设计语言编写程序, 最后调试程序, 使之运行后能产生预期的结果。程序设计语言是用于书写计算机程序的语言。语言的基础是一组记号和一组规则。根据规则, 由记号构成的记号串的全体就是语言。程序设计语言有两个方面的构成要素, 即语法和语义。语法表示程序的结构或形式, 即表示构成程序设计语言的各个符号之间的组合规律, 但不涉及这些符号的特定含义, 也不涉及使用者。语义表示程序的含义, 即表示按照各种方法所表示的各个符号的特定含义, 但不涉及使用者。到目前为止, 程序设计语言的发展经过了机器语言、汇编语言、高级语言、第四代语言四个阶段, 每一个阶段都使程序设计的效率大大提高。影响较大、使用较普遍的高级语言有C、Java、Python等。1.第一代: 机器语言
来源: uploads/第1章 Python语言概述.pdf
元数据: {'page':0, 'source': 'uploads/第1章 Python语言概述.pdf'}

2. 相关性得分: 0.9176
内容: C、Java、Python等。1.第一代: 机器语言 机器语言(machine language)是用二进制代码表示的计算机能直接识别和执行的一种机器指令的集合。它是计算机的设计者通过计算机的硬件结构赋予计算机的操作功能。机器语言具有灵活、直接执行和速度快等特点。不同型号的计算机, 其机器语言是不相通的, 即按照某种计算机的机器指令编制的程序, 不能在另一种计算机上执行。机器语言的程序难学、难写, 其编程效率极低。现在机器语言已经逐渐被淘汰了。2. 第二代: 汇编语言 汇编语言(assembly language)是一种用于电子计算机、微处理器、微控制器或其他可编程器件的低级语言, 也称为符号语言。汇编语言指令是机器指令的符号化, 与机器指令基本上存在着——对应的关系, 所以汇编语言存在难以学习、易出错、难以维护等缺点。但汇编语言也有自身的优点, 如可直接访问系统接口。3. 第三代: 高级语言 高级语言(high level language)是一种独立于机器、面向过程或对象的语言。它是较接近自然语言和数学公式的编程语言, 基本脱离了机器的硬件系统, 让人们以更易理解的方式
来源: uploads/第1章 Python语言概述.pdf
元数据: {'page':0, 'source': 'uploads/第1章 Python语言概述.pdf'}

3. 相关性得分: 0.8291
内容: 近自然语言和数学公式的编程语言, 基本脱离了机器的硬件系统, 让人们以更易理解的方式来编写程序。高级语言与计算机的硬件结构及指令系统无关, 有强的表达能力, 可方便地表示数据的运算和程序的控制结构, 能更好地描述各种算法, 而且容易学习和掌握。但高级语言编译生成的程序代码一般比用汇编语言设计的程序代码要更长, 执行的速度也更慢。高级语言的种类繁杂, 可以从应用特点和对客观系统的描述两个方面对其进行进一步分类。4. 第四代语言 第四代语言(fourth generation language, 4GL)的特征是完成一个任务仅需要告诉计算机要做什么, 而不用告诉计算机如何一步一步地做。数据库查询和应用程序生成器是第四代语言的两个典型应用。第四代语言虽然功能强大, 但在整体能力上却与高级语言有一定的差距, 目前的应用领域也比较有限。
来源: uploads/第1章 Python语言概述.pdf
元数据: {'page':0, 'source': 'uploads/第1章 Python语言概述.pdf'}

4. 相关性得分: 0.2286
内容: 1.2 Python语言的发展历史 吉多·范罗苏姆, 1956年出生于荷兰, 是Python语言的创始人。1989年, 他在ABC语言的基础上开发了一个新的脚本解释程序, 随即开始编写Python语言的编译/解释器。1991年, 第一个Python编译器诞生。它同时也是解释器, 是用C语言来实现的, 并能够调用C语言的库文件。2000年, Python2.0版本的发布吸引了越来越多的公司和个人开始使用Python; 2008年, Python3.0版本被发布。从此, Python语言有了更好的基础来向前迈进。特别要注意的是, Python3.0不向后兼容Python2.0, 这意味着Python3.0可能无法直接运行Python2.0的代码; 而且Python2.0目前已停止维护, 这也代表着Python3.0时代正式来临。Python语言的设计哲学是“优雅、明确、简单”。Python开发者的哲学理念是“用一种方法, 最好是只有一种方法来做一件事”。在设计Python程序时, 如果面临多种选择, Python
来源: uploads/第1章 Python语言概述.pdf
元数据: {'page':1, 'source': 'uploads/第1章 Python语言概述.pdf'}

5. 相关性得分: 0.2102
内容: 编译器的源代码是完全开放的, 开发者可以通过下载它学习其中的内容, 以此提高开发者的实力。(3)可移植性: python语言的可移植性是指利用python语言编写的程序可以在不做任何改动的情况下在所有主流计算机的操作系统上执行。换句话说, 在Linux下开发的一个Python程序, 如果需要在Windows系统下执行, 只要简单地吧代码复制粘贴过来, 在Windows系统安装了Python编译器后就可以很流畅地执行。可移植性正是各种平台的用户都喜欢Python语言的重要原因。Python程序已经能成功地在Linux、Windows、MacOS等许多平台上执行, 在这些平台间相互移植。(4)高级语言: 每一代编程语言的出现都会将计算机科技提升到崭新的高度, 如根据C语言, 人们开发了C++语言、Java语言这样的现代编译语言。这些语言都有高级的数据结构
来源: uploads/第1章 Python语言概述.pdf
元数据: {'page':1, 'source': 'uploads/第1章 Python语言概述.pdf'}

程序设计语言的发展经历了四个阶段:

1.第一代: 机器语言。这是最早的一种程序设计语言, 用二进制代码表示计算机能直接识别和执行的一种机器指令的集合。它具有灵活、直接执行和速度快等特点, 但难学、难写, 编程效率极低。现在机器语言已经逐渐被淘汰。

2. 第二代: 汇编语言。这是一种用于电子计算机、微处理器、微控制器或其他可编程器件的低级语言, 也称为符号语言。汇编语言指令是机器指令的符号化, 与机器指令基本上存在着——对应的关系, 所以汇编语言存在难以学习、易出错、难以维护等缺点。

3. 第三代: 高级语言。这是一种独立于机器、面向过程或对象的语言, 基本脱离了机器的硬件系统, 让人们以更易理解的方式近自然语言和数学公式的编程语言, 基本脱离了机器的硬件系统, 让人们以更易理解的方式来编写程序。高级语言与计算机的硬件结构及指令系统无关, 有强的表达能力, 能更好地描述各种算法, 而且容易学习和掌握。

4. 第四代语言: 数据库查询和应用程序生成器语言。这是第四代语言的一个典型应用, 特征是完成一个任务仅需要告诉计算机要做什么, 而不用告诉计算机如何一步一步地做。数据库查询和应用程序生成器是第四代语言的两个典型应用。

以上是程序设计语言的发展历程。

请开始提问吧!

图7 Q&A 系统简单知识问答示例

Chunks。前3个Chunks与提问的内容高度相似, 系统将相关性得分在0.8以上的检索内容融合为上下文, 注入提示词{context}, 然后调用大语言模型生成答案并根据问答模板显示内容。较之回答内容, 课程答案更为详细, 比如包含额外句子和具体语言示例, 不过二

者内容极其相似,而且回答内容的结构颇为清晰,与非知识库的大语言模型相比,该系统的回答更为贴合教学内容。

复杂编程问答示例如图 8 所示,学生分别提到实现求解凯撒密码加密程序和满足大于输入数的最小斐波那契数的程序,系统不但给出了完整的程序实例,还附带程序功能的解读。系统提供的代码逻辑清晰、结构规范,注释与代码功能也高度相符,显著增强了示例

基于LLMs和RAG的Python程序设计智能问答系统

本地大语言模型:
MiniCPM4-8B

本地嵌入模型:
MiniCPM-Embedding

本地重排序模型:
MiniCPM-Reranker

编写一个凯撒密码加密程序,接收用户输入的文本和密钥k,对明文中的字母a和字母A替换为其后第k个字母。接收两行输入,第一行为待加密的明文,第二行为密钥k。输出加密后的密文。输入样例:在这里给出一组输入,例如: Hello World! 3。

基于检索的知识库内容,无法直接提供答案,但我可以指导你如何编写这个程序。以下是一个Python实现的示例:

```
def caesar_cipher_encrypt (plain_text, key):  
    encrypted_text = ""  
  
    for char in plain_text:  
        if char.isalpha():  
            ascii_offset = 65 if char.isupper() else 97  
            encrypted_char = chr((ord(char) - ascii_offset + key) % 26 + ascii_offset)  
            encrypted_text += encrypted_char  
        else:  
            encrypted_text += char  
    return encrypted_text  
  
# 接收用户输入  
plain_text = input()  
key = int(input())  
  
# 加密并输出密文  
encrypted_text = caesar_cipher_encrypt (plain_text, key)  
print(encrypted_text)
```

这段代码定义了一个caesar_cipher_encrypt函数,它接收明文和密钥作为参数,并返回加密后的密文。代码中还包含了处理非字母字符(如空格、标点符号等)的逻辑,确保它们在加密过程中保持不变。你可以将这段代码复制到 Python 环境中运行,并输入示例中的"Hello World! 3",以查看加密效果。

斐波那契数,亦称之为斐波那契数列,指的是这样一个数列: 1、1、2、3、5、8、13、21、.....,这个数列从第3项开始,每一项都等于前两项之和。求大于输入数的最小斐波那契数。在一行输入一个正整数n(n>=10)。在一行输出大于n的最小斐波那契数。输入样例:在这里给出一组输入,例如: 10。输出样例:在这里给出相应的输出,例如: 13。

基于检索的知识库内容,无法直接提供答案,但我可以指导你如何编写这个程序。以下是一个Python实现的示例:

```
def fibonacci (n):  
    # 初始化斐波那契数列的前两项  
    a, b = 1, 1  
    # 循环生成斐波那契数列,直到找到大于输入数的最小斐波那契数  
    while b <= n:  
        a, b = b, a + b  
    # 返回大于输入数的最小斐波那契数  
    return b  
  
# 输入一个正整数n  
n = int(input())  
# 调用函数计算大于n的最小斐波那契数  
result = fibonacci(n)  
# 输出结果  
print(result)
```

这段代码定义了一个fibonacci函数,它接收输入数n作为参数,并返回大于输入数的最小斐波那契数。代码通过迭代生成斐波那契数列(仅用变量a和b动态更新),循环在当前数小于等于输入数n时持续,一旦生成大于n的数立即终止并返回结果。你可以将这段代码复制到Python环境中运行,并输入示例中的"10",以查看输出结果。

请开始提问吧!

图 8 Q&A 系统复杂编程问答示例

的可理解性与可复用性。同时,系统详细阐述了从算法逻辑到代码实现的推理过程,有效建立了从具体代码到抽象知识的认知桥梁。可见,该系统的回答不仅保障了解答的准确性以及可解释性,也体现了系统对学生深层认知需求的响应能力。

3.2 系统性能评估

利用 BLEU^[24]、ROUGE-L^[25] 和准确率指标评估系统在复杂问答任务中的性能。BLEU 是一种用于评估机器生成内容与参考内容之间相似程度的指标,其目标侧重于机器生成内容的准确性以及精确匹配程度。ROUGE-L 是一种用于评估机器生成内容涵盖参考内容完整性的指标,它是 ROUGE 的一种具体形式,其目标侧重于机器生成内容的完整性以及涵盖参考内容的程度。

对于系统在准确率方面的情况,将知识库中的客观题,如选择题、判断题以及填空题,随机抽取一部分,通过计算系统生成上述题目的正确答案所占比例的平均值来实现准确率的评估。

系统的性能表现如图 9 所示。BLEU 达到了 62.8%, ROUGE-L 为 68.5%, 准确率是 78.5%, 显然,将大语言模型、检索增强生成技术和本地化领域知识库融合的智能问答系统使得生成内容的准确性得以明显提升。

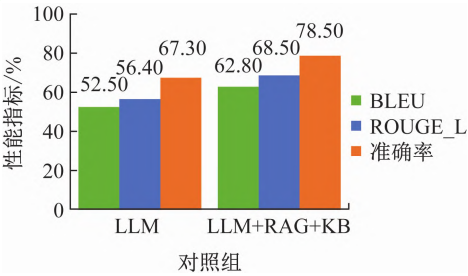


图 9 系统性能表现

3.3 辅助效果评估

为探究系统在程序设计教学中的辅助效果,采用问卷(见表 3)方式,从系统使用体验、核心能力提升和辅助学习接受 3 个维度,对学生群体开展调查评估。

问卷的信效度检验结果见表 4。问卷整体量表 Cronbach's $\alpha > 0.8$, 且 KMO > 0.7 , Bartlett 检验 $P < 0.05$, 这表明问卷的信效度是可以被接受的,数据具备一定的可靠性。

问卷结果如图 10 所示,可见,学生对该平台在辅助学习方面的接受度较高(均值 3.14),说明大部分学生对该系统是肯定的;大部分学生认为该平台的体验度也不错(均值 4.00),说明该平台在学生使用交互过程中表现友好;更为关注的是,该平台在提升学生核心能力方面具有较大优势(均值 4.18)。

表 3 辅助效果评估问卷

维度	问题	分值范围
系统使用体验度	Q11:提问和追问等交互过程是顺畅的	0 ~ 5
	Q12:生成的代码在逻辑性和解释性方面表现良好	0 ~ 5
	Q13:在理解问题意图方面表现较好	0 ~ 5
	Q14:回答内容与教材内容具有很好一致性	0 ~ 5
核心能力	Q15:系统使用体验总体满意	0 ~ 5
	Q21:算法思维能力得到锻炼	0 ~ 5
	Q22:独立解决复杂编程问题的能力得到提高	0 ~ 5
	Q23:提高了动手编程实践能力	0 ~ 5
提升度	Q24:并非全盘接受系统给出的代码和建议,批判能力有所提升	0 ~ 5
	Q25:自主学习与知识迁移能力有所提升,有利于程序设计核心概念的理解与应用	0 ~ 5
	Q26:辅助调试与错误排查有利于培养系统思维与问题分解能力	0 ~ 5
	Q27:逻辑分析与创新能力得到提升	0 ~ 5
辅助学习接受度	Q31:在编程教育中利大于弊	0 ~ 5
	Q32:有学术诚信风险	0 ~ 5
	Q33:会减少与教师的交流,不利于学习	0 ~ 5
	Q34:会产生依赖,降低自主思考和试错的意愿	0 ~ 5

表 4 问卷信效度分析

维度	α	KMO	P
系统使用体验度	0.885 5	0.847 3	0.001 4
核心能力提升度	0.890 4	0.914 6	0.041 9
辅助学习接受度	0.825 6	0.792 8	0.024 5
问卷整体量表	0.840 2	0.862 0	0.000 1

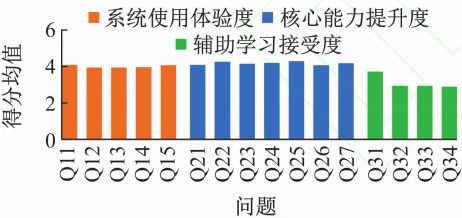


图 10 系统辅助效果

4 结 语

本文提出“动态知识 - 混合检索 - 解释生成”协同框架,探索安全可控的大模型智能辅助问答系统的构建路径,验证了 LLM 与 RAG 融合范式在高等教育场景下的应用可行性,能够有效助力提升学生知识建构、问题分析与自主学习能力。受现有数字化课程资源制约,本系统未能实现知识库实时动态增量更新,后续将重点研究知识库动态增量更新机制,进一步提升智能问答输出内容的时效性。

参考文献 (References):

[1] 舒文韬,李睿潇,孙天祥,等. 大型语言模型:原理、实现与发展[J]. 计算机研究与发展, 2024, 61(2): 351-361.

[2] 任海玉,刘建平,王 健,等. 基于大语言模型的智能问答系统研究综述[J]. 计算机工程与应用, 2025,61(7): 1-24.

[3] Chang Y P, Wang X, Wang J D, *et al.* A survey on evaluation of large language models[J]. ACM Transactions on Intelligent Systems and Technology, 2024,15(3): 1-45.

[4] 文 森,钱 力,胡懋地,等. 基于大语言模型的问答技术研究进展综述[J]. 数据分析与知识发现, 2024,8(6):16-29.

[5] 刘泽垣,王鹏江,宋晓斌,等. 大语言模型的幻觉问题研究综述[J]. 软件学报, 2025,36(3):1152-1185.

[6] 王一岩,郑永和. 智能时代的人机协同学习:价值内涵、表征形态与实践进路[J]. 中国电化教育, 2022(9):90-97.

[7] 肖建力,黄星宇,姜 飞. 智慧教育中的大语言模型综述[J]. 智能系统学报,2025,20(5):1054-1070.

[8] Ji Z W, Nayeon L, Rita F, *et al.* Survey of hallucination in natural language generation[J]. ACM Computing Surveys, 2023, 55(12): 1-38.

[9] Huang L, Yu W J, Ma W T, *et al.* A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions[J]. ACM Transactions on Information Systems, 2025, 43(2): 1-55.

[10] Zhang Y, Li Y F, Cui L Y, *et al.* Siren's song in the ai ocean: A survey on hallucination in large language models[J]. Computational Linguistics, 2025, 51(4): 1373-1418.

[11] Kazlaris I, Antoniou E, Diamantaras K, *et al.* From Illusion to Insight: A taxonomic survey of hallucination mitigation techniques in LLMs[J]. AI, 2025, 6(10): 260.

[12] Zhu K L, Luo Y F, Xu D L, *et al.* RAGEval: Scenario specific rag evaluation dataset generation framework [C]//Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL). Vienna, Austria: Association for Computational Linguistics, 2025: 8520-8544.

[13] Xiao S T, Liu Z, Zhang P T, *et al.* C-Pack: Packed resources for general Chinese embeddings [C]//Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR). Washington DC, USA: Association for Computing Machinery, 2024: 641-649.

[14] Han P, Zhou S L, Yu J, *et al.* Personalized re-ranking for recommendation with mask pretraining [J]. Data Science and Engineering, 2023, 8(4), 357-367.

[15] 张浩然,郝文宁,靳大尉,等. DF-RAG:基于查询重写和知识选择的检索增强生成方法[J]. 计算机科学, 2025,52(11):30-39.

[16] Fan W Q, Ding Y J, Ning L B, *et al.* A survey on RAG meeting LLMs: Towards retrieval-augmented large language models [C]//Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD). Barcelona, Spain: Association for Computing Machinery, 2024: 6491-6501.

[17] Brown A, Roman M, Devereux B. A systematic literature review of retrieval-augmented generation: Techniques, metrics, and challenges [J]. Big Data and Cognitive Computing, 2025, 9(12): 320.

[18] Jeong J, Gil D, Kim D, *et al.* Current research and future directions for off-site construction through langchain with a large language model [J]. Buildings, 2024, 14(8): 2374.

[19] 张鹤译,王 鑫,韩立帆,等. 大语言模型融合知识图谱的问答系统研究[J]. 计算机科学与探索, 2023,17(10):2377-2388.

[20] 潘耀宗,刘 凯,于柯远,等. 基于检索增强生成的计算机实验指导平台设计与实践[J]. 实验技术与管理, 2025, 42(4):

213-219.

[21]

徐梓斌,王亚萍,邵雅斌. Python 程序设计基础教程[M]. 2 版. 北京:北京邮电大学出版社,2023.

[22]

刘雪颖,云 静,李 博,等. 基于大型语言模型的检索增强生成综述[J]. 计算机工程与应用, 2025,61(13):1-25.

[23]

周 洁,王东毅,代沁泉,等. 生成式 AI 对话中的提示词策略有效性探究[J]. 数据分析与知识发现, 2025,9(9):49-59.

[24]

Chen A, Stanovsky G, Singh S, *et al.* Evaluating question answering evaluation [C]//Proceedings of the 2nd Workshop on Machine Reading for Question Answering (MRQA 2019). Hong Kong, China: Association for Computational Linguistics, 2019: 119-124.

[25]

Akter M, Bansal N, Karmaker S K. Revisiting automatic evaluation of extractive summarization task: Can we do better than ROUGE? [C]//Findings of the Association for Computational Linguistics (ACL 2022). Dublin, Ireland: Association for Computational Linguistics, 2022: 1547-1560.